

实验十四 自主存储控制实验

一. 实验目的和要求

1. 掌握定义事务的方法，进一步理解事务的概念和隔离性
2. 了解 MySQL 封锁技术的实现，理解封锁类型及作用
3. 了解隔离级别在 MySQL 中的应用，掌握隔离级别的设置方法

二. 实验内容

在“学生管理”数据库中完成以下操作。

1. 定义事务，理解 COMMIT 语句和 ROLLBACK 语句的作用

1) 默认事务的执行

例 1: 在 course 表中插入 3 门新课程信息

```
INSERT INTO course VALUES ('04010107','软件工程','考试',36,2,NULL);
INSERT INTO course VALUES ('04010108','机器学习','考试',40,2.5,NULL);
INSERT INTO course VALUES ('04010109','人工智能基础','考试',48,3,NULL);
```

如果最后一条语句中包含语法错误，同时执行这 3 条插入语句，查看语句的执行情况，分析执行结果。

如果最后一条语句中包含语义错误，同时执行这 3 条插入语句，查看语句的执行情况，分析执行结果。

2) 显式定义事务

例 2: 把例 1 中的 3 条正确的插入语句定义在一事务中，先后以 ROLLBACK 和 COMMIT 两种不同方式结束事务，并在事务结束后查询 course 的更新情况，说明 ROLLBACK 和 COMMIT 的不同。

```
START TRANSACTION;

INSERT INTO course VALUES ('04010107','软件工程','考试',36,2,NULL);
INSERT INTO course VALUES ('04010108','机器学习','考试',40,2.5,NULL);
INSERT INTO course VALUES ('04010109','人工智能基础','考试',48,3,NULL);
ROLLBACK;

START TRANSACTION;

INSERT INTO course VALUES ('04010107','软件工程','考试',36,2,NULL);
INSERT INTO course VALUES ('04010108','机器学习','考试',40,2.5,NULL);
INSERT INTO course VALUES ('04010109','人工智能基础','考试',48,3,NULL);
COMMIT;
```

2. 事务的隔离级别

MySQL 中默认隔离级别是 REPEATABLE READ (可重复读)。

查看当前隔离级别:

```
SELECT @@transaction_isolation;
```

```
SELECT @@global.transaction_isolation, @@session.transaction_isolation;
```

设置隔离级别为 COMMITTED-READ(读提交):

```
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

```
SET @@session.transaction_isolation = 'READ-COMMITTED';
```

例 3: 执行下表中两个事务, 观察两个事务在 COMMITTED READ 隔离级别下并发执行的情况, 分析操作的封锁情况、以及事务的并发性和数据的不一致性问题。

时间	事务 T1	事务 T2
①	START TRANSACTION; SELECT * FROM SCORE;	
②		START TRANSACTION; UPDATE SCORE SET SCORE_GRADE=SCORE_GRADE+2 WHERE STUDENT_ID='2021094001'; SELECT * FROM SCORE;
③	SELECT * FROM SCORE;	
④		COMMIT;
⑤	SELECT * FROM SCORE;	
⑥	COMMIT;	

同时打开 2 个客户端, 在一个客户端执行事务 T1, 在另一个客户端执行事务 T2。

- ① 开始执行事务 T1, 对 score 进行查询, 观察查询结果。
- ② 开始执行事务 T2, 修改学号为 2021094001 的学生的成绩并查询更新结果, 观察操作是否可以执行, score 表中元组的成绩值是否被更新?
- ③ 继续运行事务 T1, 再次执行与前面相同的查询操作, 查询是否可以执行? T1 是否会读取“脏”数据?
- ④ 提交事务 T2
- ⑤ 继续运行事务 T1, 再次执行与前面相同的查询操作, 查询是否可以执行? 查询结果中学号 2021094001 学生的成绩值与前一次查询是否一致? T1 是否出现了不可重复读现象?
- ⑥ 提交事务 T1 后, 重复⑤中的操作, 结果如何?

例 4: 执行下表中 3 个事务, 观察一个在 REPEATABLE READ(可重复读)隔离级别下的事务, 与两个在默认隔离级别 READ COMMITTED(读提交)下的事务并发执行的情况, 分析操作的封锁情况以及事务的并发性和数据的不一致性问题。

时间	事务 T1	事务 T2	事务 T3
①	START TRANSACTION; SELECT * FROM SCORE WHERE STUDENT_ID='2021094001';		

②		START TRANSACTION; SELECT * FROM SCORE WHERE STUDENT_ID='2021094002'; UPDATE SCORE SET SCORE_GRADE=SCORE_GRADE+3 WHERE STUDENT_ID='2021094001'; SELECT * FROM SCORE WHERE STUDENT_ID='2021094002'; UPDATE SCORE SET SCORE_GRADE=SCORE_GRADE+4 WHERE STUDENT_ID='2021094001';	
③	SELECT * FROM SCORE WHERE STUDENT_ID='2021094001';		
④			START TRANSACTION; INSERT INTO SCORE VALUES ('2021094001', '0401010 7', 80, '202220231'); SELECT * FROM SCORE WHERE STUDENT_ID='2021094001'; COMMIT;
⑤	SELECT * FROM SCORE WHERE STUDENT_ID='2021094001'; COMMIT;		
⑥		SELECT * FROM SCORE WHERE STUDENT_ID='2021094001'; COMMIT;	

同时打开 3 个客户端，分别在各个客户端依次执行事务 T1、T2 和 T3。

- ① 在默认隔离级别下开始执行 T1，对 score 表中学号 2021094001 的记录进行查询，观察查询结果。
- ② 首先设置事务的隔离级别为 READ COMMITTED（读提交），执行 T2，修改学号 2021094002 学生的成绩，修改前后分别查询满足更新条件的元组，比较更新前后的查询结果，查看更新是否可以实现。然后修改学号 2021094001 学生的成绩，操作能否执行？
- ③ 继续执行 T1，重新对 score 中学号 2021094001 的元组进行查询，操作是否可执行？查询结果与前次查询是否一致？
- ④ 执行 T3，向 score 表中插入一条 2021094001 的选课元组，操作是否可以执行？提交 T3
- ⑤ 继续执行 T1，重新对 score 表中 2021094001 的元组进行查询，操作是否可执行？查询结果与前次查询是否一致？是否可看到 T3 插入的元组，存在幻影现象？提交 T1
- ⑥ 继续执行 T2，重新对 score 表中 2021094001 的元组进行查询，操作是否可执行？查询结果与前次查询是否一致？是否可看到 T3 插入的元组，存在幻影现象？